



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



ASIGNATURA MACHINE LEARNING

Actividad Autónoma 3

Nombre de la actividad: Modelos lineales y no lineales en Machine Learning

Unidad 2: Modelos avanzados de clasificación y regresión

Tema 1: Modelos lineales y no lineales



FACULTAD DE
Ingeniería

Nombres:

Fecha:

Carrera:

Periodo académico:

Semestre:

Objetivo de la actividad: Aplicar modelos lineales y no lineales (regresión lineal, polinómica, regularizada, regresión logística, Softmax, SVM y redes neuronales básicas) en Google Colab, comprendiendo sus resultados, limitaciones y aplicaciones.

Recursos o temas que debe haber estudiado antes de hacer la actividad:

- Regresión lineal, polinómica y regularizada.
- Regresión logística y Softmax.
- Máquinas de Soporte Vectorial (SVM).
- Redes neuronales básicas (Perceptrón multicapa).
- Conceptos de sobreajuste y subajuste.

Formato de entrega: PDF

Instrucciones:

1. **Cree un cuaderno en Google Colab** llamado: AA1_ML_Nombre_Apellido_Grupo_X.ipynb.

- Sustituya *Nombre*, *Apellido* y *X* por sus datos reales.
- Ejemplo: AA1_ML_Castro_Juan_Grupo_2.ipynb.

2. **Copie y ejecute los códigos base provistos en la guía.**

- Antes de cada bloque de código, escriba un comentario en texto (Markdown en Colab) indicando qué modelo está implementando (ejemplo: "Regresión lineal vs polinómica").
- Dentro de cada celda de Python, añada comentarios (#) explicando la función de cada instrucción.

3. **Analice los resultados obtenidos:**

- Para cada modelo, interprete los gráficos, métricas o valores obtenidos.
- No se limite a ejecutar el código: debe escribir una explicación de qué significan los resultados en términos de ajuste, desempeño y limitaciones.

4. **Gráficos y visualización:**

- Incluya gráficos comparativos (por ejemplo, curvas de regresión, fronteras de decisión).
- Use colores y leyendas para diferenciar los modelos.

5. **Responda las preguntas planteadas en cada apartado:**

- **Regresión lineal y polinómica:** compare los MSE y explique el dilema sesgo–varianza.
- **Ridge y Lasso:** analice los coeficientes y explique cuándo conviene cada regularización.
- **Regresión logística y Softmax:** interprete el reporte de clasificación e identifique la clase con más errores.
- **SVM y Redes Neuronales:** compare accuracies, describa/dibuje la frontera de decisión y discuta ventajas y limitaciones.

6. **Organización del documento:**

- Mantenga un orden claro en el cuaderno: título, código, resultados, interpretación.
- Destaque las respuestas finales en un recuadro rojo en el PDF exportado.

7. **Formato de entrega:**

- Exporte su cuaderno desde Google Colab como **PDF:** Archivo → Descargar → Descargar como PDF.
- Nombre el archivo PDF con el mismo formato: AA1_ML_Nombre_Apellido_Grupo_X.pdf.

8. **Entrega en aula virtual:**

- Suba el archivo PDF en el espacio correspondiente de la plataforma.
- Verifique que el contenido se vea claramente y que no haya secciones en blanco.

9. **Criterios de evaluación:**

- Se calificará tanto la **ejecución correcta del código** como la **interpretación crítica de resultados**.
- La presentación y la originalidad también forman parte de la nota (ver rúbrica).

Contenido

1. Regresión lineal vs polinómica

- Ejecute el código provisto para generar los datos

$$y = \sin(x) + \text{ruido}$$

- Observe la gráfica resultante: los puntos negros representan los datos originales, la línea azul la regresión lineal y la línea roja la polinómica de grado 5.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error

# Generar datos
np.random.seed(0)
X = np.linspace(0, 10, 30).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(scale=0.3, size=X.shape[0])

# Regresión lineal
lin_reg = LinearRegression().fit(X, y)
y_pred_lin = lin_reg.predict(X)

# Regresión polinómica
poly = PolynomialFeatures(degree=5)
X_poly = poly.fit_transform(X)
poly_reg = LinearRegression().fit(X_poly, y)
y_pred_poly = poly_reg.predict(X_poly)

# Gráfica
plt.scatter(X, y, color="black", label="Datos")
plt.plot(X, y_pred_lin, label="Lineal", color="blue")
plt.plot(X, y_pred_poly, label="Polinómica grado 5", color="red")
plt.legend()
plt.show()
```

- Calcule el **error cuadrático medio (MSE)** para ambos modelos utilizando `mean_squared_error`.

```
from sklearn.metrics import mean_squared_error

mse_lin = mean_squared_error(y, y_pred_lin)
mse_poly = mean_squared_error(y, y_pred_poly)

print("MSE Lineal:", mse_lin)
print("MSE Polinómica:", mse_poly)
```

- a. Reportar ambos MSE encerrados en un recuadro rojo en el PDF.
- b. Explicar:
 - ¿Cuál de los dos modelos se ajusta mejor a los datos?
 - ¿Qué riesgo aparece cuando aumentamos el grado del polinomio (sobreajuste)?
 - ¿Cómo se relaciona esto con el dilema **sesgo-varianza**?

2. Regularización (Ridge y Lasso)

- Ejecute el código para generar un dataset con 10 variables correlacionadas.
- Observe en la salida cómo Ridge mantiene todos los coeficientes pequeños, mientras que Lasso tiende a llevar algunos coeficientes a cero.

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=100, n_features=10, noise=10,
                      random_state=0)

ridge = Ridge(alpha=1.0).fit(X, y)
lasso = Lasso(alpha=0.1).fit(X, y)

print("Coeficientes Ridge:", ridge.coef_)
print("Coeficientes Lasso:", lasso.coef_)
```

- a. Reportar los coeficientes obtenidos.
- b. Explicar:
 - ¿Cuál modelo conserva más variables en el ajuste?
 - ¿Por qué Lasso puede considerarse un método de selección de variables?
 - ¿En qué tipo de problemas conviene usar Ridge y en cuáles Lasso?

3. Regresión logística y Softmax

- Use el dataset Iris de scikit-learn.
- Ejecute el modelo de regresión logística con max_iter=200.
- Observe el reporte de clasificación que incluye precisión, recall y F1-score por clase.

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

iris = load_iris()
X, y = iris.data, iris.target

log_reg = LogisticRegression(max_iter=200).fit(X, y)
y_pred = log_reg.predict(X)

print(classification_report(y, y_pred,
                             target_names=iris.target_names))
```

- a. Analizar el reporte de clasificación.
- b. Identificar:
 - ¿Qué clase fue más fácil de clasificar y por qué?
 - ¿Qué clase presenta más errores?
 - ¿Qué métrica (precisión, recall o F1) sería más importante en un contexto real de biología o medicina?

4. SVM y Redes Neuronales

- Use el dataset `make_moons` de `scikit-learn` para generar datos con dos clases en forma de media luna.
- Entrene un modelo SVM con kernel RBF y un perceptrón multicapa (MLP) con 5 neuronas ocultas.
- Reporte la accuracy de ambos modelos.

```
from sklearn.datasets import make_moons
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier

X, y = make_moons(n_samples=200, noise=0.2, random_state=0)

svm = SVC(kernel="rbf", C=1, gamma=0.5).fit(X, y)
mlp = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000,
random_state=0).fit(X, y)

print("Accuracy SVM:", svm.score(X, y))
print("Accuracy MLP:", mlp.score(X, y))
```

- a. Reportar la accuracy de cada modelo.
- b. Dibujar (o describir) cómo imagina la frontera de decisión de ambos modelos.
- c. Explicar:
 - ¿Cuál modelo resulta más flexible y por qué?
 - ¿Qué ventaja tiene la SVM frente a la red neuronal?
 - ¿En qué tipo de dataset sería preferible usar un MLP en lugar de una SVM?



Bibliografía:

- Benítez, R., Cencerrado, A., Escudero, G., & Kanaan, S. (2025). Inteligencia artificial avanzada. FUOC.
- Géron, A. (2023). Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow (3rd ed.). O'Reilly.
- Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2023). Applied Logistic Regression (4th ed.). Wiley.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). An Introduction to Statistical Learning with Applications in Python (2nd ed.). Springer.
- Müller, A. C., & Guido, S. (2017). Introduction to Machine Learning with Python: A Guide for Data Scientists. O'Reilly Media.
- Raschka, S., Liu, Y., & Mirjalili, V. (2022). Machine Learning with PyTorch and Scikit-Learn. Packt.
- scikit-learn developers. (2024). User Guide v1.4–1.5 (Linear models; SVM; Neural networks; Model selection).



Rúbrica de evaluación

Componente de aprendizaje:	Autónomo	X	Contacto con el Docente	
Nombre de la Unidad:	Unidad 2: Modelos avanzados de clasificación y regresión			
Resultado(s) de aprendizaje:	El estudiante aplicará modelos avanzados supervisados en problemas reales.			
Nombre de la Actividad:	Modelos lineales y no lineales en Machine Learning			

Criterios de Evaluación	Escala de Valoración						
	Excelente (10 - 9,1)	Bueno (9 - 8,1)	Satisfactorio (8 - 7)	Necesita mejorar (6,9 - 0,1)	No entrega (0)	Puntaje	Comentarios (SIGEA)
1. Modelado lineal y polinómico	Ejecuta código correctamente; calcula MSE; interpreta sesgo-varianza con claridad y ejemplos.	Ejecuta código y calcula MSE; explicación aceptable.	Ejecuta código parcialmente o con errores; explicación básica.	Resultados incompletos o sin interpretación adecuada.	No entrega		
2. Regularización (Ridge y Lasso)	Reporta coeficientes y explica con precisión las diferencias; analiza utilidad práctica de cada modelo.	Reporta coeficientes y comenta diferencias generales.	Resultados parciales o explicación limitada.	Presenta resultados incorrectos o sin análisis.	No entrega		
3. Clasificación (Regresión logística y Softmax)	Presenta reporte completo; identifica correctamente clases fáciles/difíciles de clasificar; interpreta métricas en contexto real.	Presenta reporte y explica resultados de forma general.	Reporte incompleto o análisis superficial.	Resultados erróneos o sin interpretación.	No entrega		
4. SVM y Redes Neuronales	Reporta accuracies; describe/dibuja fronteras de decisión; analiza ventajas y limitaciones de ambos modelos.	Reporta accuracies y comenta diferencias generales.	Resultados parciales; análisis limitado.	Presenta resultados sin justificación.	No entrega		
5. Presentación, autonomía y reflexión crítica	Documento claro, ordenado, sin plagio; refleja autonomía, innovación y alineación a ejes formativos (ética, comunicación, tecnologías).	Documento ordenado y comprensible; autonomía básica.	Documento aceptable pero con poca reflexión crítica.	Documento desordenado, sin reflexión ni autonomía.	No entrega		

Puntaje total 50

